FACULTAD DE INFORMÁTICA
MÁSTER DE INGENIERÍA DEL SOFTWARE



SolidarianID

Memoria de prácticas

Grupo 3:
Antonio Pérez Serrano
Fabián Sola Durán
Manuel Bernabé Rodríguez
Pablo Peris Solano
Sergio Ros Liarte

Desarrollo Full-Stack
12/01/2026

Índice

1. Introducción **3**

2. Arquitectura general de la solución **3**
2.1. Aplicación principal (MVP) basada en microservicios . . . 3
2.2. Aplicación de administración: monolito MVC con SSR . . . 3
2.3. Integración entre aplicaciones y flujo de datos . . . 4

3. Backend **5**
3.1. Organización de los módulos del monorepo . . . 5
3.1.1. Microservicio de usuarios: `apps/users/` . . . 5
3.1.2. Microservicio de comunidades: `apps/communities/` . . . 6
3.1.3. Microservicio de administración: `apps/admin/` . . . 6
3.1.4. Biblioteca compartida: `libs/` . . . 7
3.1.5. Configuración del monorepo . . . 7
3.2. Diagramas del modelo del dominio . . . 7
3.3. Criterios aplicados para la organización del código del proyecto. . . . 9
3.3.1. Principios *SOLID* . . . 10
3.3.2. *Clean Architecture* y arquitectura hexagonal . . . 10
3.3.3. *Domain-Driven Design (DDD)* . . . 10
3.3.4. Convenciones de Nomenclatura . . . 11
3.3.5. Gestión de configuración del proyecto . . . 11
3.3.6. Documentación del código . . . 11
3.4. Aplicación de arquitectura limpia y hexagonal . . . 12
3.5. Estrategia de control de errores . . . 13

4. Frontend **14**
4.1. Partials compartidos entre vistas . . . 15
4.1.1. Sistema de Design Tokens con Tailwind CSS . . . 15
4.1.2. Aside (Barra lateral de navegación) . . . 16
4.1.3. Footer (Pie de página) . . . 17
4.2. Paneles de login y registro . . . 18
4.2.1. Controladores . . . 18
4.2.2. Validación de datos . . . 18
4.2.3. Lógica de autenticación . . . 19
4.2.4. Gestión de sesiones y seguridad . . . 19
4.2.5. Accesibilidad . . . 19
4.3. *Dashboard* de métricas . . . 20
4.4. Panel para gestión de solicitudes de creación . . . 21
4.5. Informes de usuarios . . . 21
4.5.1. PDFKit . . . 21
4.6. Patrón de delegación de eventos . . . 22

5. Conclusiones **23**

A. Introducción a SolidarianID **26**

Índice de figuras

1. Arquitectura de contenedores (C2) del MVP de SolidarianID: microservicios de *Users* y *Communities* y aplicación de administración monolítica. 4
2. Modelo de dominio del microservicio de usuarios 8
3. Modelo de dominio del microservicio de administración 8
4. Modelo de dominio del microservicio de comunidades 9
5. Componente reutilizable *aside* 26
6. Componente reutilizable *footer* 27
7. Pantalla de login 28
8. Pantalla de registro 29
9. Panel de estadísticas de la aplicación 30
10. Panel de validación de solicitudes de comunidades 31
11. Panel de validación de solicitudes de causas 31
12. Panel de validación de solicitudes de acciones 32
13. Cuadro de diálogo para introducir el motivo 32
14. Panel de informes de usuario 33
15. Vista detallada de usuario 34
16. Página de errores con Not Found 35

1. Introducción

En esta memoria abarcaremos los puntos más importantes del diseño y la implementación software de SolidarianID, el MVP de la aplicación solidaria que engloba el desarrollo de las prácticas del primer cuatrimestre del Máster en Ingeniería del Software. En concreto, ahondaremos en la arquitectura general que hemos diseñado para la aplicación, los módulos o microservicios en los que se divide y los principios de diseño que han guiado el desarrollo.

En cuanto al segundo requisito de esta práctica, la aplicación MVC de administración para evaluar la parte de *frontend* de la asignatura, indicaremos más detalladamente la implementación concreta del *frontend* y cómo se han completado los requisitos que solicitados (Tailwind, DaisyUI, PDFKit, Paginación, Handlebars...).

En la bibliografía se detalla el enlace [1] a nuestro repositorio de GitHub en el cual se pueden encontrar las distintas carpetas de código fuente (`*/src/*`) que hemos llevado a cabo.

2. Arquitectura general de la solución

En el MVP de *SolidarianID* hemos diseñado una arquitectura compuesta por **dos aplicaciones principales** con responsabilidades claramente diferenciadas: (i) la **aplicación principal** basada en **microservicios** (expuesta por un API REST), y (ii) la **aplicación de administración** implementada como un **monolito MVC** que integra **backend y frontend** mediante *Server-Side Rendering* (SSR). Esta separación nos permite evolucionar el *core* funcional de la plataforma de forma independiente del panel administrativo, manteniendo además límites claros de acceso y despliegue.

2.1. Aplicación principal (MVP) basada en microservicios

La aplicación principal del MVP se implementa como un conjunto de **APIs independientes** desarrolladas con NestJS y organizadas en un *monorepo*, donde cada aplicación representa un contexto delimitado del dominio. En particular, el MVP expone:

- **Microservicio de usuarios** (`apps/users/`): encargado del registro, autenticación y gestión del perfil de usuario. Expone una API REST (documentada con Swagger) y publica eventos de dominio cuando se producen cambios relevantes.
- **Microservicio de comunidades** (`apps/communities/`): responsable de la gestión de comunidades y del contenido asociado (causas, acciones, eventos, objetivos), así como del flujo de solicitudes de creación que serán revisadas desde administración.

Desde el punto de vista de interacción, los usuarios (registrados y no registrados) acceden a la plataforma mediante **peticiones HTTPS** a estas APIs. Tal como se ilustra en la Figura 1, cada microservicio mantiene su responsabilidad y su ciclo de vida, favoreciendo el desacoplamiento y la evolución incremental del sistema.

2.2. Aplicación de administración: monolito MVC con SSR

En paralelo, se ha desarrollado una **aplicación de administración** como un **monolito NestJS** con patrón **MVC**, orientado a las necesidades del **administrador de la plataforma**: moderación y validación de solicitudes, auditoría e informes, y consulta de métricas en el dashboard.

A diferencia de los microservicios (centrados en API), este monolito integra **backend y frontend** en la misma aplicación. El **renderizado de vistas** se realiza mediante:

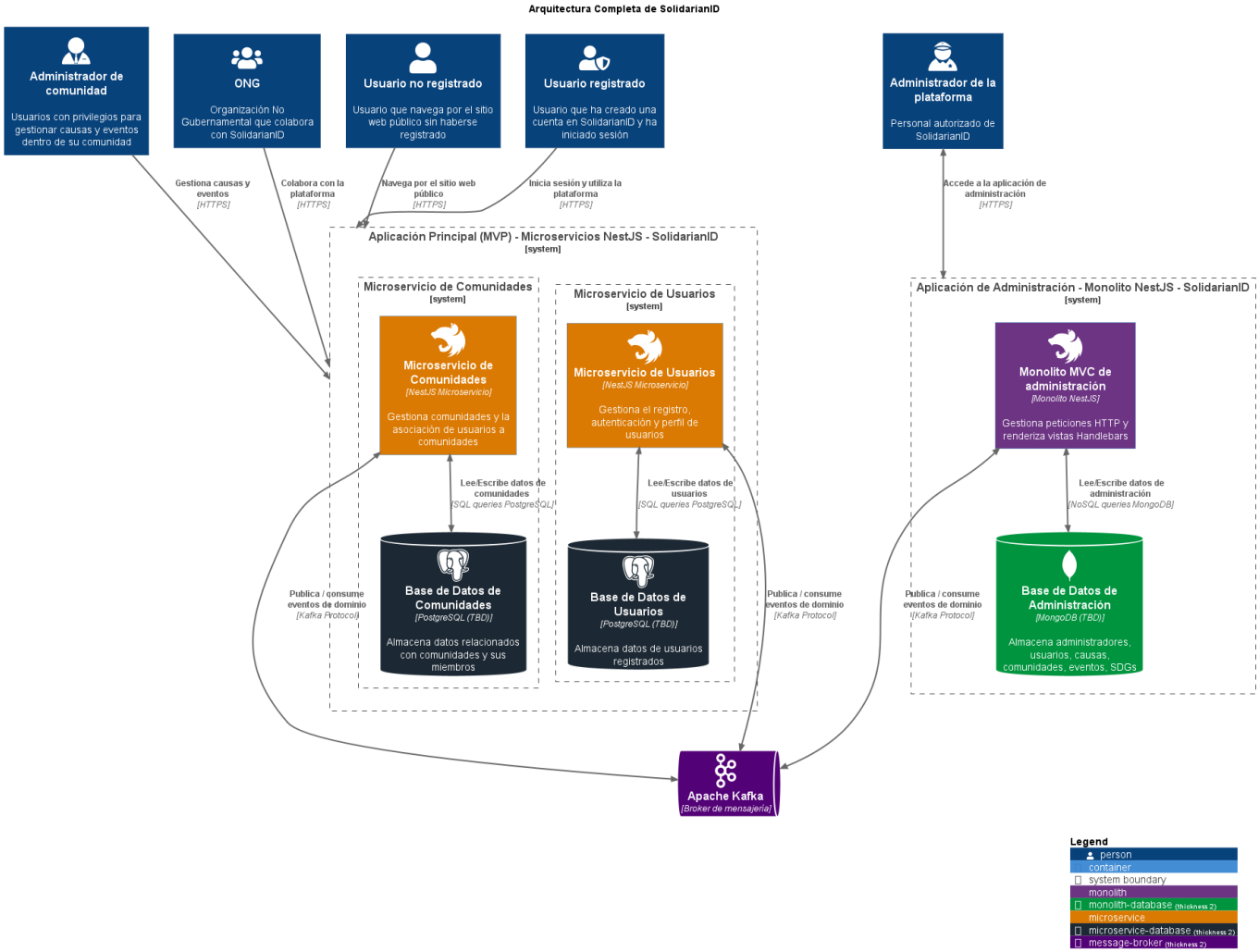


Figura 1: Arquitectura de contenedores (C2) del MVP de SolidarianID: microservicios de *Users* y *Communities* y aplicación de administración monolítica.

- **Express** + **NestJS** como servidor HTTP.
- **Handlebars** como motor de plantillas para generar las vistas en el servidor (SSR).
- Estructura de vistas organizada en `views/` (layouts/pages/partials) y recursos estáticos en `public/` (CSS/JS/iconos).

Este enfoque resulta especialmente adecuado para un panel administrativo, ya que simplifica el despliegue (una única aplicación), centraliza la navegación y permite controlar el acceso mediante mecanismos de sesión de Express para administradores.

2.3. Integración entre aplicaciones y flujo de datos

Aunque existen dos aplicaciones diferenciadas, el sistema se concibe como una única solución coherente:

- La **aplicación principal** proporciona el *core* de negocio y expone APIs para el consumo de la plataforma.
- La **aplicación de administración** consume y presenta información de interés para tareas de moderación y análisis, renderizando vistas SSR para el personal autorizado.

Tal y como se recoge en la memoria del proyecto, la comunicación entre componentes se apoya en **eventos** para sincronizar información relevante entre contextos (por ejemplo, solicitudes o cambios de usuarios) y evitar dependencias estrictamente síncronas entre servicios, manteniendo así un mayor desacoplamiento y escalabilidad evolutiva de los aplicativos por separado. Por otro lado, en la carpeta raíz del repositorio se encontrarán las consultas postman asociadas para facilitar el testeo de la aplicación.

3. Backend

En este apartado veremos nuestra organización del código, los distintos microservicios en los que hemos organizado el código y las interacciones que realiza por debajo el backend del MVP de *SolidarianID*.

3.1. Organización de los módulos del monorepo

El proyecto *SolidarianID* se ha desarrollado utilizando una arquitectura de *monorepo* en *NestJS*, lo que permite gestionar múltiples aplicaciones y bibliotecas compartidas dentro de un único repositorio de código fuente. Esta decisión arquitectónica facilita la reutilización de código, la consistencia en las convenciones de desarrollo y la gestión centralizada de dependencias.

El *monorepo* se estructura en tres grandes bloques:

- **Aplicaciones (`apps/`)**: Contiene tres proyectos independientes pero interconectados (*admin*, *communities* y *users*).
- **Bibliotecas compartidas (`libs/`)**: Proporciona funcionalidades transversales reutilizables por todas las aplicaciones.
- **Configuración del testing `test/`**: Configuraciones generales para *testing* (*jest*)

A continuación detallaremos la organización de las tres aplicaciones: *admin*, *communities* y *users*, cada una con responsabilidades claramente diferenciadas:

3.1.1. Microservicio de usuarios: `apps/users/`

Microservicio encargado de la gestión del dominio de usuarios y sus perfiles. Implementa Arquitectura Hexagonal (*Ports and Adapters*) combinada con *Domain-Driven Design (DDD)*. Se sigue la siguiente estructura:

- `domain/`: Contiene la lógica de negocio pura, independiente del *framework*
  - `model/`: Entidades de dominio, agregados y objetos de valor (*value-objects*, siguiendo las prácticas de *DDD*).
  - `ports/`: Interfaces (puertos) que definen contratos con la infraestructura (repositorios, *publishers*).
  - `services/`: Interfaces del servicio de la aplicación.
  - `errors/`: Excepciones personalizadas del dominio.
- `application/`: Capa de aplicación que orquesta los casos de uso:
  - `services/`: Implementaciones de las interfaces de los servicios (ubicadas en la capa de

dominio) que conectan el dominio con la infraestructura.

- `infrastructure/`: Adaptadores concretos a tecnologías específicas:
  - `persistence/`: Repositorios implementados con *TypeORM* (base de datos relacional *PostgreSQL*), incluyendo entidades de persistencia y mappers.
  - `messaging/`: Adaptadores de mensajería con *Kafka* para comunicación asíncrona (*producers*).
  - `presentation/`: Controladores *REST* con documentación *OpenAPI/Swagger*.

Esta aplicación expone una *API REST* documentada con *Swagger* y publica eventos de dominio a través de *Kafka* cuando ocurren cambios relevantes (ej: registro de usuario, actualización de perfil).

3.1.2. Microservicio de comunidades: `apps/communities/`

Microservicio responsable de la gestión de comunidades, causas, acciones, eventos solidarios y objetivos. Al igual que en el microservicio *users*, se sigue arquitectura hexagonal con *DDD*, siguiendo una estructura similar de directorios. Este microservicio también es el encargado de gestionar las solicitudes de creación de las comunidades, causas y acciones, las cuales se enviarán de forma asíncrona al microservicio de *admin* a través de *kafka*, para que este las valide o las rechace. También consume eventos del microservicio de usuarios, con el fin de gestionar las membresías a las comunidades.

3.1.3. Microservicio de administración: `apps/admin/`

Microservicio de administración que proporciona una interfaz web para los administradores de la plataforma. A diferencia de los otros dos microservicios, este implementa un patrón *MVC (Model-View-Controller)* tradicional, pero mantiene la filosofía *DDD* en su modelado de dominio. Se sigue la siguiente estructura:

- `administrator/`: Módulo principal con submódulos para gestión de usuarios, comunidades y solicitudes.
  - `modules/`: Submódulos *NestJS* encapsulados (`users.module.ts`, `communities.module.ts` y `requests.module.ts`)
  - `controllers/`: Controladores para vistas (*View Controllers*) y API REST (*API Controllers*).
  - `model/`: Pese a que en la aplicación de administración se sigue un modelo *MVC* tradicional, en este directorio se encuentra el *core* del microservicio, por lo que decidimos organizarlo siguiendo arquitectura hexagonal y *DDD*.
    - `domain/`: Servicios de dominio y puertos de salida (*repository ports*).
    - `application/`: Servicios de aplicación que implementan casos de uso.
    - `infrastructure/`: Repositorios implementados con *Mongoose* (*MongoDB*) para operaciones de consulta optimizadas.
- `auth/`: Módulo de autenticación con sesiones de *express* para administradores, siguiendo una estructura de directorios similar a la de *administrator* pero centrada solo en la autenticación.
- `types/`: Interfaz de la sesión de *express*.

- `utils/`: Pequeñas funciones y constantes utilizadas a menudo en la aplicación.

El microservicio de usuarios también escucha eventos de *Kafka* procedentes de *users* y *communities* para sincronizar datos en la base de datos de consultas.

En cuanto a las vistas también diferenciamos algunos directorios:

- `public/`: *CSS* compilado (*Tailwind*), *JavaScript* del cliente para hacer la interfaz dinámica e iconos.
- `views/`: Plantillas *Handlebars* organizadas en *layouts* (componentes fijos de la interfaz), *pages* (Contenido de las páginas) y *partials* (componentes plantilla *hbs* para simplificar las páginas).

3.1.4. Biblioteca compartida: `libs/`

El directorio `libs/` contiene bibliotecas reutilizables que promueven los siguientes principios: *DRY* (*Don't Repeat Yourself*), KISS (*Keep It Simple Stupid!*) y la cohesión del sistema. Gracias a este evitamos duplicar código común en cada microservicio.

- `ddd/`: Proporciona clases base y abstracciones para implementar *DDD*, tales como *aggregate-root*, *entity*, *value-object*, *domain-event*, *either* y los repositorios genéricos.
- `kafka/`: Encapsula la configuración y lógica de comunicación con *Apache Kafka*, tales como los módulos dinámicos para productores y consumidores; los *DTOs* y *mappers* de eventos; las enumeraciones de *topics* y eventos; y la abstracción del cliente *KafkaJS* mediante la implementación de un publicador genérico adaptado al uso que estamos haciendo de DDD.
- `utils/`: Funciones auxiliares comunes, gestión de contraseñas con *bcrypt* (*PasswordManager*) en el directorio `security/` o los *helpers* para las respuestas *HTTP* en el directorio `http/`.
- `value-objects/`: Objetos de valor compartidos entre aplicaciones (ej: *Email*, *UniqueId*, *Address*).
- `design-tokens/`: Utilidades de diseño de interfaz como por ejemplo *Tokens* de diseño (colores, tipografías, espaciados), archivos fuente de estilos (*SCSS*, *Tailwind CSS*) y configuración de *Tailwind* compartida.
- `charts/`: Biblioteca para generación de gráficos con *Chart.js*, utilizada por la aplicación *admin* para visualización de métricas.
- `test/`: Utilidades y *helpers* para *testing* (*mocks*, *fixtures*, configuraciones compartidas).

3.1.5. Configuración del monorepo

La gestión del monorepo se realiza mediante dos archivos principales, el `nest-cli.json`, que define la estructura del monorepo y la configuración de cada proyecto; y el `package.json`, que centraliza las dependencias compartidas de todas las aplicaciones y los diferentes *scripts* que se han ejecutado durante el desarrollo (compilación, *testing*, *docker*, *linting*, *prettier*, etc).

3.2. Diagramas del modelo del dominio

A continuación se muestran los 3 diagramas del modelo de dominio. Se puede observar el diagrama de usuarios en la Figura 2, el diagrama de administración en la Figura 3 y el diagrama de comunidades en la Figura 4. Estos diagramas han sido generados automáticamente a partir del propio código fuente, gracias a la herramienta *PlantUML*.

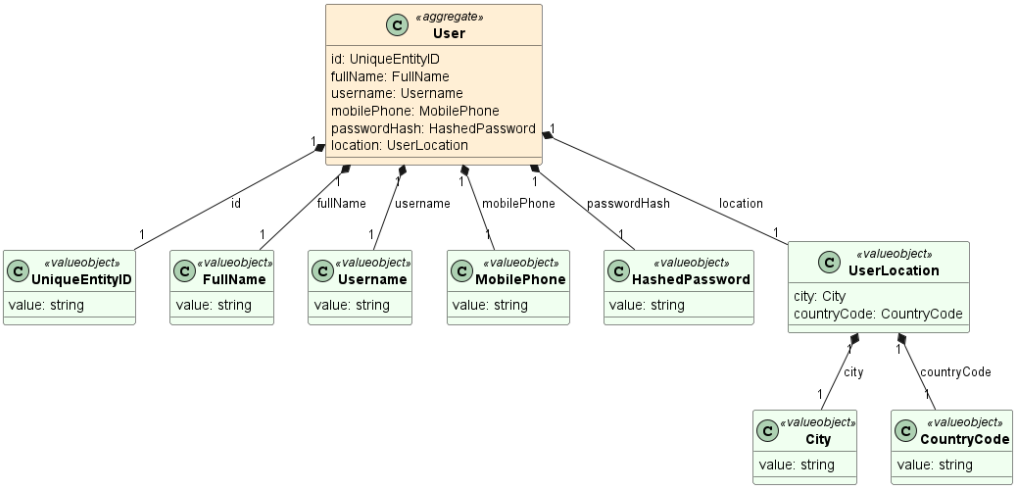


Figura 2: Modelo de dominio del microservicio de usuarios

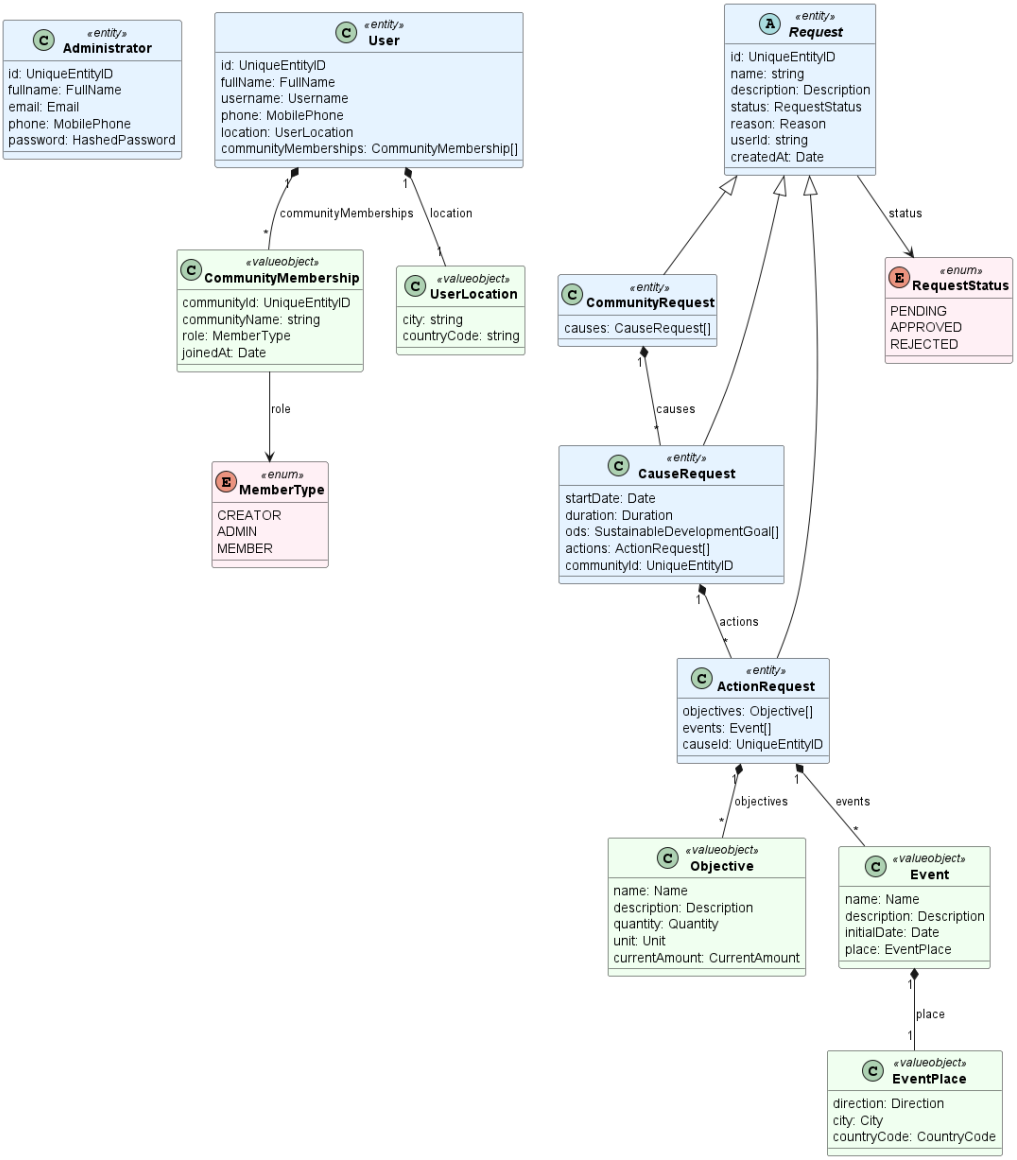


Figura 3: Modelo de dominio del microservicio de administración

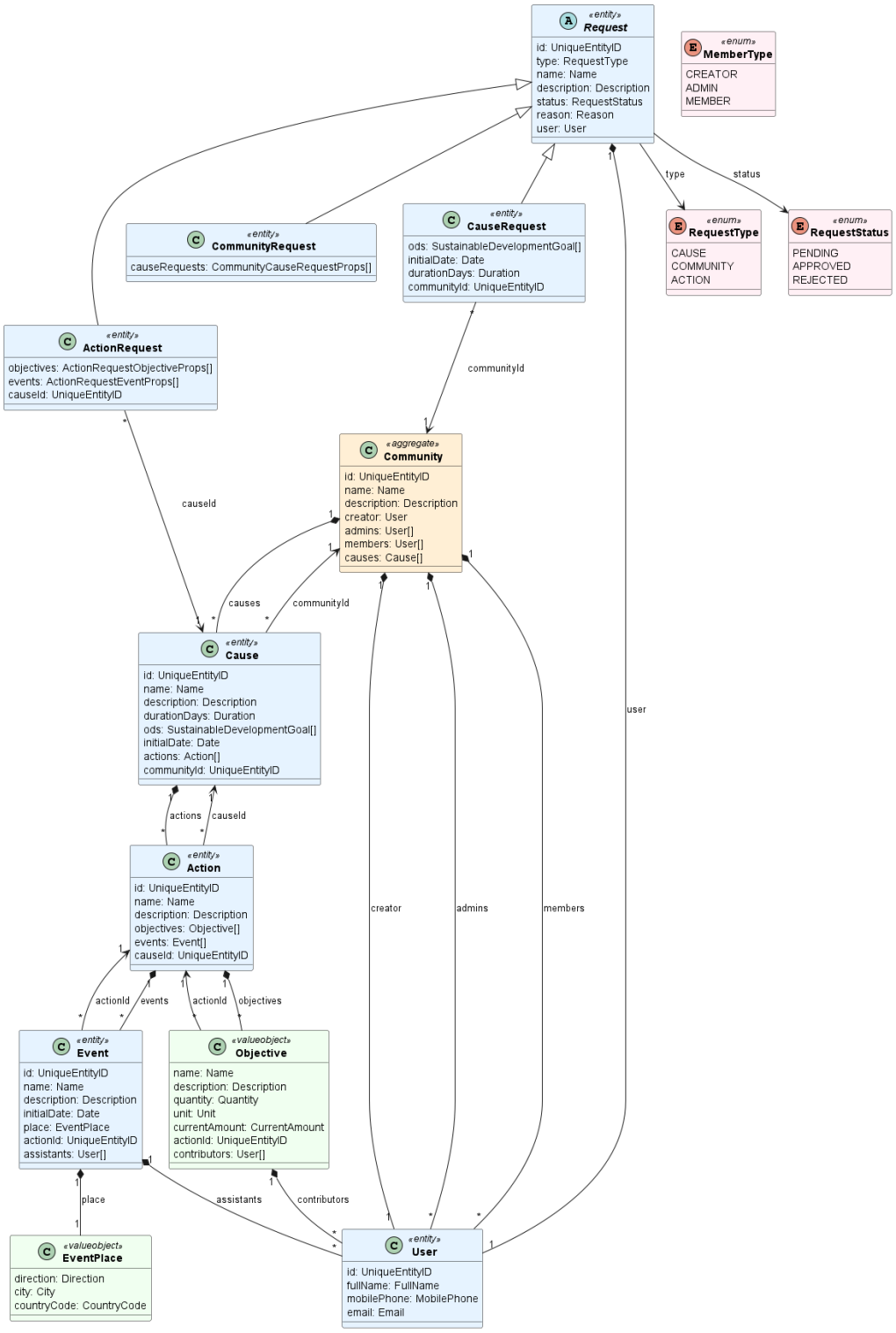


Figura 4: Modelo de dominio del microservicio de comunidades

3.3. Criterios aplicados para la organización del código del proyecto.

La organización del código del proyecto SolidarianID se fundamenta en los siguientes principios y patrones de diseño de software:

SOLID, organizacion por capas, convenciones de nomenclatura, Gestión de Configuración, Testing Organizado por Capas y Documentación del Código.

3.3.1. Principios *SOLID*

La aplicación de los principios *SOLID* busca que el código sea legible, mantenible, flexible y escalable. Para ello, se estructura el sistema de modo que cada componente tenga una única responsabilidad bien definida, se pueda extender sin modificar el código existente, y las implementaciones concretas sean intercambiables gracias al uso de interfaces específicas. Todo esto se logra haciendo uso del principio de inversión de dependencias de la siguiente manera que se describe a continuación: la capa de aplicación (lógica de negocio) depende de las abstracciones del dominio, y la de infraestructura (como persistencia o *APIs*) también se adaptan a esas abstracciones, facilitando la modificación y el *testing* sin impactar el núcleo de la aplicación.

3.3.2. *Clean Architecture* y arquitectura hexagonal

Los criterios aplicados para la organización del código se han basado en conceptos como *Clean Architecture* y arquitectura hexagonal, ya que nos permite organizar el código en capas (*domain, application and infrastructure*) con dependencias unidireccionales, ya que las capas internas nunca dependen de las externas.

Esto nos permite que la capa de dominio sea 100 % independiente del *framework*, ya que esta contiene los puertos (contratos) a los que se tendrán que adaptar las capas externas (aplicación e infraestructura). Otra decisión que tomamos para diferenciar claramente los contratos fue separar el de los servicios (`services/` para la capa de aplicación) y los de infraestructura (`/ports`).

En la capa de aplicación se implementan las interfaces de los servicios de la capa de dominio con la lógica de negocio (desarrollo de los casos de uso), coordinando las llamadas a los repositorios.

La capa de infraestructura contiene los adaptadores que implementan los puertos de la capa de dominio, con el fin de realizar la comunicación con servicios externos como bases de datos (**patrón repositorio**), mensajería asíncrona con otros microservicios (**kafka**) y controladores *REST* para la comunicación mediante *HTTP*. Para cada servicio externo también se han creado las clases pertinentes para realizar la comunicación con cada tecnología concreta, de forma que por ejemplo tenemos clases `Entity` para el acceso a base de datos y clases *DTO* para la comunicación con el cliente *HTTP*. De igual forma, se han creado las clases `Mapper` para transmitir la información entre estas clases y las de nuestro modelo de dominio.

3.3.3. *Domain-Driven Design (DDD)*

Durante el desarrollo se han aplicado conceptos de *DDD* como el de*Bounded Contexts*, donde cada aplicación (microservicio) representa un contexto delimitado con su propio modelo de dominio:

- *Users Context*: Gestión de identidades y autenticación.
- *Communities Context*: Comunidades, causas, acciones, eventos y objetivos.
- *Admin Context*: Validación de solicitudes, métricas y reportes.

Además, el código refleja el lenguaje del negocio (*Ubiquitous Language*), ya que términos como *Community*, *Cause*, *Action*, etc, provienen del dominio y los nombres de métodos expresan intenciones de negocio, como por ejemplo `validateCause()`, `registerUser()` o `publishEvent()`.

Otros conceptos importantes de *DDD* que se aplican son los *Aggregates*, *Entities* y *Value Objects*:

- ***Aggregates***: *Community* es el *AggregateRoot* por excelencia de nuestro proyecto, ya que es el encargado de agregar las causas pertenecientes a una comunidad, que a su vez contienen las acciones solidarias. Además, si por ejemplo se eliminase una comunidad, se eliminarían también

sus causas y las acciones asociadas a estas.

- ***Entities***: Las entidades son aquellas clases relevantes del modelo de dominio que requieren tener identificador para ser consultadas, modificadas, creadas o eliminadas. Algunos ejemplos serían los usuarios, las solicitudes de creación de una comunidad o las acciones solidarias.

- ***Value Objects***: Se utilizan extensivamente para encapsular lógica de validación, ya que estas clases son inmutables y se validan en el momento de creación (por ejemplo *Username*, *MobilePhone*, *Email*, etc.). Además, implementan igualdad por valor, no por identidad

Tampoco nos podemos olvidar de los *Domain Events*, que son los eventos de dominio que capturan hechos importantes del negocio, como por ejemplo la creación de un usuario (`USER_CREATED`) o la creación de una solicitud de creación (`REQUEST_CREATED`). Esto ha sido posible gracias a *Kafka* y a la definición de una serie de *topics*, tales como `USERS`, `REQUESTS`, `COMMUNITIES`, `CAUSES`, `EVENTS`, que han servido para configurar los consumidores de cada microservicio, consumiendo de donde le interesaba a cada uno.

3.3.4. Convenciones de Nomenclatura

Para que todos los miembros del equipo nos pusiéramos de acuerdo a la hora de programar, se decidió seguir convenciones de nombrado, tales como las siguientes:

- Archivos y directorios: *kebab-case* con los sufijos correspondientes (`users.service.impl.ts`). El sufijo representa la responsabilidad de la clase que hay dentro del archivo, como `.port.ts` para los puertos, o `.dto.ts` para las clases *DTO*.

- Nombre de clases e interfaces: *PascalCase*, como por ejemplo `UsersService` o `UserEntity`. Incluso en las clases pertinentes se utilizaban sufijos como `Port` y `Adapter` para representar el concepto puerto-adaptador de la arquitectura hexagonal.

- Métodos, variables y constantes: para las variables y métodos se ha utilizado *camelCase* (como `registerUser()` o `mobilePhone`) y para las constantes *UPPER_SNAKE_CASE* (como DEFAULT_PAGE_SIZE)

3.3.5. Gestión de configuración del proyecto

Cada microservicio tiene su propia configuración de módulos, ya que gracias a los ficheros `.module.ts` es posible configurar la importación de dependencias, los registros de *providers* (inyección de dependencias) y la exportación de servicios públicos. Algunos ejemplos son los módulos de cada microservicio (`UsersModule`, `CommunitiesModule` y `AdminModule`) e incluso los módulos de las tecnologías como *kafa* (`KafkaModule`) o la base de datos *SQL* (`DatabaseSqlModule`), que en nuestro caso es *PostgreSQL*.

También, un aspecto importante de la configuración del proyecto son las variables de entorno contenidas en los ficheros `.env`, que dependiendo del tipo se usan para producción (`.env.production`), desarrollo (`.env.development`) o *testing* (`.env.test`).

3.3.6. Documentación del código

Simplemente comentar que se ha decidido utilizar *Swagger/OpenAPI* para la documentación de los controladores *REST*.

3.4. Aplicación de arquitectura limpia y hexagonal

Tal como hemos comentado en el apartado 3.3.2, la arquitectura hexagonal es el patrón arquitectónico principal aplicado en los microservicios de *users* y *communities* del proyecto *SolidarianID*. Además, como bien hemos comentado anteriormente, en algunas partes del microservicio de *admin* se ha implementado también una estructura de directorios similar a la de *clean architecture* de los otros microservicios. Esta arquitectura persigue los siguientes objetivos:

- Independencia del *framework*: La lógica de negocio no debe depender de *frameworks* externos.
- *Testabilidad*: El dominio debe ser *testeable* sin necesidad de interfaz de usuario o servicios externos.
- Independencia de la *UI*: La interfaz de usuario puede cambiar sin afectar la lógica de negocio.
- Independencia de la base de datos: La *BD* es un detalle de implementación intercambiable.
- Independencia de agentes externos: La lógica de negocio no conoce el mundo exterior.

Además, esta arquitectura también es conocida como **puerto-adaptador**. Los puertos son interfaces abstractas definidas en la capa de dominio que establecen contratos para comunicarse con el exterior. En cambio, los adaptadores son implementaciones concretas en la capa de infraestructura que conectan los puertos con tecnologías específicas, como por ejemplo las bases de datos (*PostgreSQL*) o con *Kafka*. Destacar también que para el acceso a las bases de datos se ha utilizado el patrón repositorio, tanto para bases de datos relacionales con *TypeORM* como para *MongoDB* con *mogoose*, además de que en la librería compartida se ha implementado un repositorio genérico para cada una, con el fin de que todos los microservicios accedan de forma homogénea a *BD*. Por otro lado, las interfaces de los servicios también se ubican en la capa de dominio, y estas son implementadas en la capa de aplicación, adaptándose al contrato de dichas interfaces.

En nuestro proyecto *SolidarianID* hay numerosos adaptadores debido a la gran variedad de tecnologías externas que utilizamos, tales como:

- **Adaptadores de Persistencia**: Utilizados para el acceso a base de datos, tales como el `UsersRepositoryTypeORM` para la base de datos *PostgreSQL* en el microservicio de usuarios o el `UserRepositoryMongoose` en *admin* para las consultas de su base de datos *MongoBD*.
- **Adaptadores de Mensajería**: Se han implementado también los adaptadores de *Kafka* con el fin de comunicar los distintos microservicios de forma asíncrona. Además, destacar que se ha implementado un publicador genérico de *Kafka* con el fin de facilitar las notificaciones de los eventos del dominio.
- **Servicios Externos**: Por otro lado, se ha implementado un adaptador para utilizar la librería *bcrypt*, con el fin de cifrar las contraseñas de los usuarios y administradores que se almacenan en base de datos.

Por otro lado, un factor importante de la arquitectura hexagonal es la inyección de Dependencias, la cual es gestionada por lo ficheros `.module.ts` de *NestJS*, gracias al *array* de *providers*, el cual relaciona cada *provide* (puerto) con su *useClass* (adaptador). Esto permite intercambiar implementaciones sin modificar el dominio, el *testing* fácil (*mockear* adaptadores inyectando implementaciones falsas) y cumplir el pincipio de inversión de dependencias, donde el dominio depende de abstracciones.

Tal como hemos comentado en el apartado 3.3.2, en la capa de infraestructura es necesario crear las clases concretas y sus respectivos *mappers* de cada tecnología, como por ejemplo las clases *entities* para las bases de datos y sus *mappers* que las transforman en clases de nuestro modelo de dominio.

Este enfoque de arquitectura hexagonal tiene muchas ventajas, tales como la posibilidad en el futuro de cambiar de base de datos (por ejemplo de *PostgreSQL* a *MySQL*), de servicio de mensajería (de *Kafka* a *RabbitMQ*) o incluso de controlador *REST* a *GraphQL*, todo sin necesidad de modificar el núcleo del código de la aplicación (dominio y aplicación).

3.5. Estrategia de control de errores

El proyecto implementa el patrón *Either* para la gestión explícita de errores a través de las capas de modelo, aplicación e infraestructura (la parte de repositorios). Este tipo aporta un enfoque funcional y representa una disyunción exclusiva: `Left<L>` encapsula errores del dominio, mientras que `Right<R>` contiene valores exitosos. A diferencia de las excepciones tradicionales con *try/catch*, *Either* hace que los errores sean parte del sistema de tipos, obligando a verificar que todos los casos se manejen explícitamente.

Los repositorios implementan puertos que retornan `Promise<Either<RepositoryException, Entity»`. Cuando ocurre una excepción técnica (como un error de conexión a base de datos), se captura y traduce a un error de dominio mediante `left(new RepositoryException(...)`, preservando la causa original para debugging. En caso de éxito, retornan `right(entity)`. Esta traducción asegura que ningún detalle de infraestructura contamina el dominio.

Las entidades del dominio (como `User`) componen múltiples *Value Objects* usando *Either*. Durante la construcción de una entidad, si algún *Value Object* falla su validación, el error se propaga automáticamente, deteniendo la creación y retornando el primer *Left* encontrado. Los repositorios traducen excepciones técnicas (errores de base de datos, conexión) a errores de dominio específicos (`UserNotFoundError, RepositoryError`) mediante *left()*, mientras que operaciones exitosas retornan *right(entity)*.

Lo mismo en los servicios de capa de aplicación como *UsersService*. Todos los métodos implementan el patrón, devolviendo con *Left* el primer error encontrado o con *Right* la entidad o el resultado esperado.

El objetivo es abordar la gestión de errores de una forma funcional, evitando el *overhead* técnico y de recursos que suponen las excepciones tradicionales. Además, el uso del patrón *Either* va acompañado de errores semánticos personalizados como `RepositoryException`, `EntityNotFoundError` o `ValidationError` que muestran de una forma más clara cual es el error ocasionado.

```
export abstract class UsersService {
    abstract loginUser(username: string, password: string):
    Promise<Either<Error, boolean>>;
}

export class User extends Entity<UserProps> {
    public static create(props: UserCreationProps, id?: string):
    Either<ValidationError, User> {
        ...
    }
}

export abstract class UsersRepositoryPort extends Repository<User> {
    abstract existsByMobilePhone(
    mobilePhone: string,
    excludeUserId?: string,
  ): Promise<Either<RepositoryException, boolean>>;
}
```

Código 1: Algunos ejemplos de uso de Eihter y errores semánticos.

4. Frontend

En esta sección, detallaremos lo necesario sobre la aplicación de administración, desarrollada para cumplir con los requisitos de la parte de *frontend* de la asignatura. En esta aplicación, se permite a los administradores de la plataforma *SolidarianID* realizar labores de moderación (aceptar o rechazar solicitudes de creación de comunidades, causas y acciones), auditoría (revisar informes sobre los usuarios de la aplicación) y estadística (con un *dashboard* para consultar métricas de uso).

La sección está estructurada de forma que veremos las distintas pantallas que forman la aplicación de administración, y dentro de cada una veremos qué parte de los requisitos de la práctica cumple.

Antes de proceder con las vistas una a una, vamos a repasar las tecnologías transversales que hemos aplicado en todas, conforme a los requisitos de la práctica:

- ***Flexbox, Grid y Media Queries***: Tanto *flex* como *grid* han sido utilizadas para hacer *layouts*, pero la que más hemos utilizado ha sido *flexbox*, ya que nos resultaba más fácil la organización de la interfaz en filas simples (o columnas). Igualmente, también hemos empleado *Grid* para diseñar el *footer* y las *Media Queries* para su diseño dependiendo del tamaño de la pantalla.

- ***Sass***: *Sass* se ha utilizado para algunos estilos en una parte de la interfaz de la página de solicitudes pendientes de comunidad. Para ello, se ha compilado el fichero *.sass* en un fichero *.css*, el cual puede interpretar el navegador.

- **Tailwind CLI y DaisyUI**: Ambas tecnologías han enriquecido mucho el desarrollo de nuestra interfaz. Hemos aplicado estilos *tailwind* en todas las interfaces, debido a que aplicar los estilos en las propiedades del propio componente resultaba muy cómodo. También hemos utilizado *DaisyUI* en la mayoría de vistas para reutilizar componentes ya hechos y estilizados, lo que ha acelerado bastante el desarrollo.

- ***Handlebars***: Todas las vistas de la aplicación (*login*, registro, *dashboard*, *reports* y *requests*) se generan a través de plantillas de *Handlebars*, renderizándose con *Server Side Rendering* en la parte del servidor.

- **Chart.js**: Esta ha sido la librería utilizada para generar los gráficos en la vista del *dashboard* del administrador, de forma que este tuviese una visión general de métricas de uso de la aplicación.
- ***Pdfkit***: Esta librería se ha utilizado para generar el *PDF* con la información asociada a un usuario.
- ***JavaScript***: Es el lengüaje que hemos usado para hacer dinámica nuestra interfaz en el navegador, y gracias a ella también se validan los datos que se introducen de un administrador durante el proceso de registro.
- ***Mobile-first***: Siguiendo el principio de diseño recomendado por *Tailwind CSS*, todas las vistas están implementadas primero con vista de teléfono móvil, de forma que no se tenga que quitar funcionalidad de una pantalla una vez ya creada por falta de espacio.
- Sesión de *Express*: Una vez que un administrador ha iniciado sesión, la aplicación mantiene una sesión de *Express* para recordar los datos del administrador autenticado.

4.1. Partials compartidos entre vistas

Con el fin de reutilizar código y mantener una estructura de vistas homogénea, se ha adoptado el uso de *layouts* en las plantillas `.hbs`. En concreto, se han definido dos *layouts* principales: (i) un *layout* orientado a las pantallas públicas de autenticación (inicio de sesión y registro), que incluye únicamente los elementos comunes mínimos, como el *footer*; y (ii) un *layout* para las vistas internas del panel de administración, que incorpora un *aside* de navegación y un *footer*. La composición y el contenido de estos elementos se detallarán en los siguientes subapartados.

4.1.1. Sistema de Design Tokens con Tailwind CSS

En el proyecto **SolidarianID**, se ha implementado un sistema modular en `/libs` llamado *design-tokens* basado en Tailwind CSS, aprovechando su arquitectura de capas (`@layer`) para organizar los estilos de manera escalable y mantenible.

El archivo principal `app.css` actúa como punto de entrada centralizado, importando tanto las directivas fundamentales de Tailwind como los módulos específicos del proyecto:

- Importa la librería base de Tailwind CSS mediante `@import 'tailwindcss'`
- Integra plugins especializados como `@iconify/tailwind4` para iconografía y `daisyui` para componentes UI preconfigurados
- Orquesta la carga secuencial de hojas de estilo modulares: `theme.css`, `components.css`, `layouts.css`, `page.css` y `requests.css`

Esta arquitectura explota el modelo de capas de Tailwind, donde:

1. **theme.css** define las variables de diseño base (paleta cromática mediante custom properties CSS, tipografías)
2. **components.css** implementa la capa `@layer components` con clases reutilizables de alto nivel (auth-container, auth-card, etc.)
3. **layouts.css** especifica elementos estructurales comunes como footer y navegación
4. Los archivos específicos (`page.css`, `requests.css`) encapsulan estilos contextuales de los partials

```
@import 'tailwindcss';
@plugin @iconify/tailwind4;
@plugin daisyui {
  themes:
    light --default,
    dark --prefersdark;
  exclude: footer, button;
}
@import './theme.css';
@import './components.css';
@import './layouts.css';
@import './page.css';
@import './requests.css';
```

Código 2: Organización de imports en app.css

4.1.2. Aside (Barra lateral de navegación)

El componente `aside` implementa la navegación principal del panel de administración. Se diseñó siguiendo un enfoque *mobile-first* con comportamiento adaptativo según el tamaño de pantalla.

Estructura HTML: El componente se divide en tres elementos principales definidos en el partial `aside.hbs`:

- **Hamburger Header**: Cabecera fija visible únicamente en dispositivos móviles que contiene el botón de menú hamburguesa y el logo de la aplicación.
- **Aside Overlay**: Capa semitransparente que oscurece el contenido principal cuando el menú lateral está desplegado en móvil.
- **Aside**: Barra lateral con el menú de navegación, que incluye enlaces al dashboard, solicitudes (con submenú desplegable), informes, registro y botón de cierre de sesión.

Comportamiento responsive: Se aplican las siguientes media queries mediante clases de Tailwind CSS:

- **Móvil (<768px)**: El aside se posiciona de forma fija fuera de la pantalla (`-translate-x-full`) y se desliza mediante una transición CSS cuando se activa la clase `.open`).
- **Escritorio (>768px)**: El aside se muestra de forma permanente con posición `sticky`, ocupando un ancho fijo en el grid del dashboard. El botón hamburguesa y el overlay se ocultan automáticamente.

Estilos CSS aplicados:

Navegación con submenús: Se utiliza el elemento nativo HTML5 `<details>` con `<summary>` para implementar menús desplegables accesibles sin necesidad de JavaScript, siguiendo las directrices de accesibilidad.

Accesibilidad:

- Etiquetas `aria-label` en botones e iconos.
- Atributos `aria-hidden=true` en iconos decorativos.

```
.aside {
  @apply fixed top-0 left-0 bottom-0 w-2/3 z-50
         -translate-x-full transition-transform duration-300
         md:sticky md:top-0 md:h-screen md:translate-x-0;
}

.aside.open {
  @apply translate-x-0;
}

.hamburger-header {
  @apply fixed top-0 left-0 right-0 z-50 md:hidden;
}
```

```
.footer-top {
  display: grid;
  grid-template-columns: 1fr;  /* Móvil: 1 columna */
  text-align: center;
}

@media (min-width: 640px) {
  .footer-top {
    grid-template-columns: 1fr 1fr;  /* Tablet: 2 columnas */
  }
}

@media (min-width: 1024px) {
  .footer-top {
    grid-template-columns: repeat(4, 1fr);  /* Desktop: 4 columnas */
  }
}
```

- Navegación semántica con `<nav aria-label=Navegación lateral>`.
- Uso de roles `img` en iconos con significado.

4.1.3. Footer (Pie de página)

El componente `footer` proporciona navegación secundaria, enlaces de ayuda e información institucional. Implementa un diseño en grid con reorganización automática según el viewport.

Estructura: El footer se organiza en dos secciones principales:

- **Footer Top**: Grid de navegación dividido en cuatro columnas temáticas: "Otros apartados", "Necesitas ayuda", "Noticias e Información" y "Otros sitios".
- **Footer Bottom**: Información de copyright y enlaces a redes sociales con iconos interactivos.

Sistema de grid responsive: Se implementa un grid progresivo mediante media queries:

Breakpoints aplicados:

- $<$**390px**: Layout de columna única con secciones apiladas.

```
.footer-bottom {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
  justify-content: space-between;
}
```

- **390px - 639px**: Se añade margen inferior a cada sección.
- **640px - 1023px**: Grid de 2 columnas para mejor aprovechamiento del espacio.
- >= **1024px**: Grid completo de 4 columnas mostrando todas las secciones en línea.

Estilos visuales: El footer utiliza la paleta de colores corporativa con fondo `ocean-blue-900` y texto blanco. Los enlaces incluyen efectos hover con cambio de fondo, y los iconos de redes sociales aplican un filtro de brillo (`hover:brightness-130`).

Accesibilidad:

- Navegación semántica con `<nav aria-label=“Navegación del pie de página”>`.
- Sección de redes sociales con `aria-label=“Redes sociales”`.
- Cada enlace de red social incluye su propio `aria-label` descriptivo.
- Iconos marcados con `aria-hidden=true` al ser puramente decorativos junto al texto alternativo.

Flexbox en Footer Bottom: La sección inferior utiliza Flexbox con `flex-wrap: wrap` para permitir que el copyright y los iconos sociales se reorganicen en pantallas pequeñas:

4.2. Paneles de login y registro

A continuación se detallan las primeras vistas implementadas en la aplicación: Inicio y registro de sesión y registro. Todas estas vistas se presentarán comentadas para poder usarse en el Anexo A

4.2.1. Controladores

Se implementaron dos controladores separados siguiendo el principio de responsabilidad única:

- **AuthViewController**: Controlador MVC que renderiza las vistas de login (`/login`) y registro (`/register`) utilizando Handlebars. La ruta de registro está protegida mediante un guard de autenticación.
- **AuthApiController**: Controlador REST bajo el prefijo `/api/auth` que expone los endpoints POST para `login`, `register` y `logout`.

4.2.2. Validación de datos

La validación se realiza en múltiples niveles:

Validación en el cliente: Los archivos `login-validation.js` y `register-validation.js` implementan validación en tiempo real utilizando la API de validación nativa del navegador y expresiones regulares personalizadas. Se validan:

- Formato de correo electrónico
- Contraseña con mínimo 12 caracteres, una mayúscula, un número y un carácter especial
- Nombre completo sin números ni caracteres especiales
- Teléfono en formato internacional (ej: +34612345678)

Validación en el servidor: Se utilizan DTOs (*Data Transfer Objects*) con decoradores de *class-validator*:

- `AdminLoginRequestDto`: Valida email y contraseña para el inicio de sesión.
- `AdminRegisterRequestDto`: Valida todos los campos necesarios para el registro incluyendo nombre completo, email, teléfono y contraseña.

4.2.3. Lógica de autenticación

El servicio `AuthServiceImpl` implementa la lógica de negocio:

Registro:

1. Verifica que no exista un administrador con el mismo email.
2. Hashea la contraseña utilizando `PasswordManager` que por debajo utiliza BCrypt para encriptar la contraseña.
3. Crea la entidad `Administrator` mediante el patrón Factory Method.
4. Persiste el administrador en MongoDB a través del repositorio.

Login:

1. Busca el administrador por email en la base de datos.
2. Compara la contraseña proporcionada con el hash almacenado.
3. Almacena los datos del usuario en la sesión HTTP.
4. Retorna un DTO con la información del administrador (sin contraseña).

4.2.4. Gestión de sesiones y seguridad

La autenticación se gestiona mediante sesiones HTTP. El `AuthGuard` implementa la interfaz `CanActivate` de NestJS para proteger rutas que requieren autenticación, verificando la existencia de `request.session.user`

4.2.5. Accesibilidad

Las vistas implementan las directrices WCAG 2.2 Nivel AA:

- Etiquetas ARIA (`aria-label`, `aria-describedby`, `aria-required`)
- Roles semánticos (`role=“alert“` para mensajes de error)
- Regiones `aria-live` para anuncios dinámicos

- Navegación por teclado en todos los elementos interactivos
- Indicadores visuales de estado de validación con iconos accesibles

4.3. *Dashboard* de métricas

En el proyecto **SolidarianID**, se ha implementado una arquitectura modular y tipada para la generación de gráficos analíticos utilizando Chart.js como se puede ver en la figura 9 del Anexo.

Librería de Construcción de Gráficos (`libs/charts`)

Se ha creado una librería reutilizable ubicada en `solidarian-id/libs/charts/src/` utilizadas en la librería *Char.js* que encapsula la configuración de cuatro tipos de gráficos:

- **Pie Charts** (`pie.ts`): Mediante la función `createPieConfig()`, genera configuraciones para gráficos circulares con soporte para callbacks de tooltips serializables
- **Doughnut Charts** (`doughnut.ts`): A través de `createDoughnutConfig()`, permite configurar gráficos de anillo con parámetro ajustable de cutout
- **Bar Charts** (`bar.ts`): Con `createBarConfig()`, soporta barras horizontales/verticales y opciones de leyenda
- **Line Charts** (`line.ts`): La función `createLineConfig()` habilita gráficos multipropósito con múltiples series y control de tensión de curva

Cada función retorna un objeto de tipo `SerializableChartConfig`, que extiende la configuración estándar de Chart.js añadiendo la propiedad `_serializedCallbacks` para permitir la serialización de funciones JavaScript como strings (requisito para transferir configuraciones desde el backend NestJS hacia el frontend como los tooltips y hovers).

Capa de Negocio: Chart Builder (`apps/admin/src/utils/chart-builder.ts`)

Este módulo actúa como *factory* de alto nivel, transformando DTOs del dominio en configuraciones de gráficos específicas del contexto de negocio:

1. `createTopCommunitiesPie()`: Genera un gráfico circular mostrando las comunidades con mayor número de usuarios
2. `createInteractiveNationalitiesDoughnut()`: Produce un gráfico de anillo con distribución de usuarios por nacionalidad
3. `createOdsByCauseBar()`: Crea un gráfico de barras verticales mapeando causas solidarias a los 17 Objetivos de Desarrollo Sostenible (ODS)
4. `createMonthlyJoinsLine()`: Construye un gráfico lineal comparativo de adhesiones mensuales por comunidad

Capa de Presentación

Vista Handlebars (`apps/admin/views/pages/metrics.hbs`): Define cuatro elementos `<canvas>` accesibles (WCAG 2.2 Level AA) con identificadores únicos:

- `chart-communities`, `chart-nationalities`, `chart-ods`, `chart-joins`

En esta vista recibe serializado las gráficas que se rehidratan con los callbacks para que chart detecte los tooltips.

4.4. Panel para gestión de solicitudes de creación

Esta interfaz corresponde con la gestión de solicitudes pendientes que le llegan a un administrador, desde la cual podrá tanto ver sus detalles como validarlas o rechazarlas. Además, cuando un administrador pinche en la sección *Solicitudes* de la barra lateral, se le desplegará un menú donde podrá seleccionar el tipo de solicitudes que quiere gestionar: comunidades, causas o acciones. Este menú puede observarse en la barra lateral de la Figura 10.

Cabe destacar que tanto las solicitudes de comunidades, como las de causas y acciones se cargan como listados paginados (siendo el total el número que hay bajo la cabecera), con el fin de no sobrecargar la vista cuando hay muchas solicitudes almacenadas en base de datos. Esto quiere decir que, cuando hay más de 10 solicitudes del mismo tipo, el administrador tendrá la capacidad de navegar entre las distintas páginas mediante los botones de navegación que aparecerán al final de todas las solicitudes.

Por otro lado, tal como podemos observar en la Figura 10, el administrador podrá validar o rechazar las solicitudes de comunidades. Una vez le de al botón, aparecerá un cuadro de diálogo para que el administrador pueda escribir un motivo de la decisión tomada.

Tanto los botones de validar y rechazar las solicitudes como el cuadro de diálogo para introducir el motivo están creados con plantillas reutilizables `.hbs`, de forma que es mucho más escalable, ya que todas las solicitudes utilizan estos mismos componentes. Simplemente en su fichero *javascript* correspondiente hemos asociado un *listener* que escuche cuando se hace clic sobre esos botones, de forma que se ejecuta la función pertinente en cada caso. El cuadro de diálogo se explica más en detalle en el anexo, en la figura 13.

La mayoría de estilos de esta vista se han hecho con *tailwind* y componentes *daisy*, como por ejemplo la lista de *ODS* de las causas asociadas a las comunidades, los botones de paginación o los botones de validar y rechazar. También se ha utilizado *sass* para hacer que estos botones se coloquen en la esquina de cada solicitud de creación. Por último, se ha utilizado *CSS* para elegir los colores de los contenedores de los *items* (comunidades, causas, color de los bordes, etc).

4.5. Informes de usuarios

El objetivo de esta página es el último panel solicitado en la práctica, en el que se pueden ver todos los usuarios registrados en la aplicación y solicitar un informe con los datos de cada uno. Como vemos en la Figura 14, es una **vista paginada** , como se solicitaba, y sigue una paginación tradicional en lugar de AJAX, donde el número de página se trata en la URL de la página como un parámetro (`?page=3`).

Además, la página incluye en su Javascript un "*drawer*" que hace una petición a la API de la aplicación y recupera toda la información de un solo usuario, mostrándola a través del DOM para que el administrador pueda comprobarla. Podemos verla en la Figura 15.

4.5.1. PDFKit

En el mismo *drawer* que vemos en la Figura 15, podemos ver el botón "Descargar PDF". Este, como su propio nombre indica, hace una petición a la API de la aplicación. El endpoint `/users/<id>/pdf` devuelve en su cuerpo un objeto de tipo `application/pdf` con un reporte del usuario. En el anexo a este documento podemos ver un documento generado mediante este servicio.

Para realizar la implementación de este servicio de descarga de PDFs hemos utilizado la librería

PDFKit. Para ello, hemos creado un puerto de servicio `UserPdfGeneratorPort` de arquitectura hexagonal, con el que hemos realizado un adaptador para PDFKit, guardando los patrones de diseño de nuestra aplicación. En el Código 3 podemos ver como hemos usado el tipo `PDFDocument` de PDFKit para renderizar un PDF en base a métodos privados para generar las secciones.

```
@Injectable()
export class PdfKitUserReportGenerator implements UserPdfGeneratorPort {
  generate(data: UserReportPdfCommand): Either<PdfGenerationException, Readable> {
    try {
      const doc = new PDFDocument({
        size: 'A4',
        margins: { top: 50, bottom: 50, left: 50, right: 50 },
        info: {
          Title: `Reporte de Usuario - ${data.username}`,
          Author: 'SolidarianID',
          Subject: 'Reporte de Usuario',
          Keywords: 'reporte, usuario, voluntarios',
          CreationDate: data.generatedAt,
        },
      });
      this.renderHeader(doc, data);
      this.renderPersonalInfo(doc, data);
      this.renderLocation(doc, data);
      this.renderCommunities(doc, data);
      this.renderFooter(doc, data);
      doc.end();
      return right(doc as unknown as Readable);
    } catch (error) {
      ...
    }
    ...
  }
```

Código 3: Método principal del generador de PDF.

4.6. Patrón de delegación de eventos

El patrón de delegación de eventos (*Event Delegation*) se utiliza para optimizar la gestión de eventos en listas dinámicas, evitando la asignación de *listeners* individuales a cada elemento. Este patrón lo aplicamos en muchas vistas de la aplicación, como por ejemplo en la validación de solicitudes, tal como se puede observar en el trozo de código 4.6:

```
    document.addEventListener('click', e => {
      const button = e.target.closest('button[data-action]');
      if (!button) return;

      const article = button.closest('article.request');
      if (!article) return;

      const id = article.dataset.id;
      const action = button.dataset.action;
```

```
    showReasonModal(id, action);
  });
```

Como vemos en el código, este patrón es eficiente debido a que un solo *listener* en el *document* maneja *clicks* en todos los botones validar/rechazar de todas las solicitudes, ya que no importa si hay 10 o 1000 solicitudes, el *listener* es único.

Además, como hemos dicho este patrón también se ha utilizado en otras partes de las vistas, tales como en los reportes de usuarios (pinchar en tarjetas de usuarios) o incluso en la barra lateral (pinchando en los *links*).

El no aplicar el patrón de delegación de eventos genera múltiples problemas. En rendimiento, consume excesiva memoria (hasta 195 veces más), ya que cada *listener* individual ocupa memoria, mientras que la delegación usa uno solo. También degrada páginas largas al ralentizar la inicialización, ya que bloquea el *thread* principal al asignar eventos uno por uno. Además, presenta problemas con contenido dinámico, ya que los nuevos elementos añadidos no tendrán *listeners* automáticamente, obligando a re-ejecutar selectores y *bindings*.

Por lo que estamos viendo este patrón resulta imprescindible en cualquier aplicación *frontend*, ya que sin este se generaría código más verboso y aumentaría el riesgo de *memory leaks* si no se limpian los *listeners* adecuadamente.

5. Conclusiones

El desarrollo de esta práctica ha consolidado una visión integral del *Full-Stack* aplicada a un caso de estudio realista: *SolidarianID*. A lo largo del trabajo hemos diseñado e implementado una solución completa que combina una aplicación principal basada en microservicios (*users* y *communities*) con una aplicación de administración en forma de monolito MVC con *Server-Side Rendering*. Este enfoque nos ha permitido comprender, desde la práctica, cómo separar responsabilidades a nivel de arquitectura y cómo articular la interacción entre servicios orientados a API y una interfaz web administrativa centrada en productividad y control.

En el ámbito del **backend**, el mayor impacto formativo ha sido interiorizar patrones y criterios de diseño propios del desarrollo profesional: organización en monorepo, separación por capas mediante *Clean Architecture* / arquitectura hexagonal, y modelado con *Domain-Driven Design* en distintos *bounded contexts*. Además, el uso de una biblioteca compartida (`libs/`) para reutilización de abstracciones de DDD, utilidades y componentes transversales ha reforzado la importancia de la cohesión, el control de dependencias y la consistencia del código en un sistema que crece por módulos. Especial mención merece la estrategia de control de errores mediante el patrón *Either*, que nos ha aportado una disciplina adicional en el tratamiento explícito de fallos, reduciendo ambigüedades y favoreciendo un flujo de control más predecible entre capas.

En cuanto al **frontend**, la implementación del panel de administración nos ha permitido afianzar competencias clave para la construcción de interfaces mantenibles: composición de vistas con Handlebars, renderizado SSR, gestión de sesión de administrador, diseño *mobile-first*, maquetación con Flex/Grid y uso de herramientas modernas como Tailwind y DaisyUI para acelerar el desarrollo sin perder consistencia visual. La integración de *Chart.js* para métricas y de *PDFKit* para generación de informes en PDF nos ha aportado experiencia práctica en la creación de funcionalidades habituales en entornos empresariales (dashboards, reporting), además de ejercitar la conexión efectiva entre datos, endpoints y representación en UI.

A nivel de **ingeniería del software**, este trabajo ha reforzado habilidades fundamentales: capacidad de justificar decisiones técnicas, diseño orientado a extensibilidad, organización del código en equipos,

y documentación de APIs con Swagger/OpenAPI.

Como líneas de mejora, el proyecto queda preparado para seguir ampliando funcionalidades del dominio, robustecer mecanismos de integración entre servicios, y continuar elevando el nivel de calidad mediante mayor automatización (por ejemplo, ampliación de pruebas y validaciones) y refuerzo de aspectos no funcionales conforme el sistema evolucione. En definitiva, *SolidarianID* ha servido como un entorno práctico para consolidar conocimientos técnicos y metodológicos propios del desarrollo *Full-Stack* moderno, con una base arquitectónica sólida para iterar de forma segura en futuras entregas cuando incorporemos los microservicios de gestión de red social y de mensajería entre usuarios además de la interfaz visual de la aplicación principal.

Referencias

[1] Equipo 3. *Repositorio de GitHub del equipo 3*. 2026. URL: https://github.com/fsoladur/SolidarianID.

A. Introducción a SolidarianID

Componentes Reutilizables del Sistema

El panel de administración de SolidarianID se construye mediante componentes modulares reutilizables implementados como *partials* de Handlebars. Estos componentes garantizan consistencia visual y funcional a lo largo de toda la aplicación.

Barra Lateral de Navegación

El componente `aside.hbs` (ver Figura 5) constituye el elemento de navegación principal del panel administrativo. Implementa un menú lateral persistente con las siguientes funcionalidades:

- **Navegación jerárquica**: Incluye enlaces directos a Dashboard, sistema de Solicitudes (con submenu colapsable para Comunidades, Causas y Acciones), Informes y Registro de administradores
- **Responsive design**: En dispositivos móviles se transforma en un menú hamburguesa activable mediante botón, con overlay semitransparente para mejorar la usabilidad
- **Accesibilidad**: Todos los enlaces incluyen atributos ARIA apropiados (`aria-label`, `role=ïmg"`), iconografía de Iconify, y navegación por teclado completa
- **Autenticación**: Botón de cierre de sesión situado en la parte inferior con confirmación JavaScript



Figura 5: Componente reutilizable *aside*

Pie de Página Corporativo

El *partial* `footer.hbs` (ver Figura 6) proporciona navegación secundaria y enlaces institucionales organizados en cuatro secciones temáticas: apartados principales (Métricas, Solicitudes, PDF), ayuda y soporte, noticias corporativas, y ecosistema de aplicaciones. Adicionalmente, incorpora enlaces a redes sociales (Twitter, Instagram, LinkedIn, Facebook) con iconografía vectorial y etiquetas ARIA para tecnologías asistivas. Este componente se renderiza de forma consistente en todas las vistas que utilizan el layout `dashboard.hbs`.

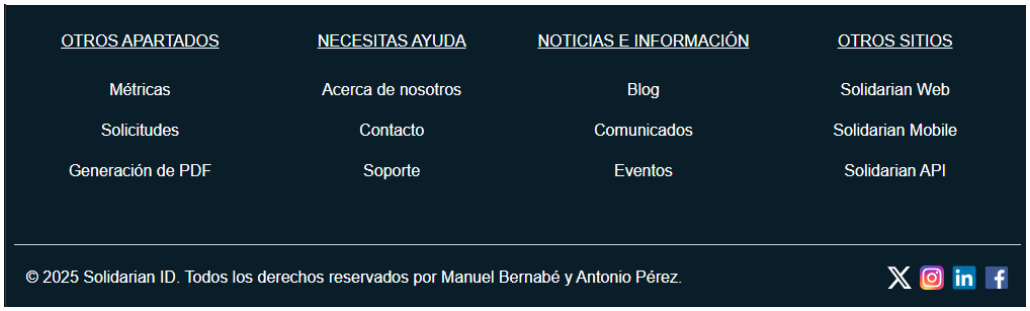


Figura 6: Componente reutilizable *footer*

Vistas de Autenticación

Inicio de Sesión

La vista `login.hbs` (ver Figura 7) implementa el formulario de autenticación para administradores del sistema. Características destacadas:

- **Validación en cliente**: Patrones regex para email y contraseña segura (mínimo 12 caracteres, mayúscula, número y carácter especial)
- **UX avanzada**: Campo de contraseña con botón toggle de visibilidad, iconos de estado de validación, mensajes de ayuda contextuales
- **Accesibilidad WCAG 2.2**: Región live `aria-live=.ªssertive"` para anunciar errores, `autocomplete` apropiado para gestores de contraseñas
- **Seguridad**: Método POST con protección CSRF (implementada a nivel de middleware NestJS)

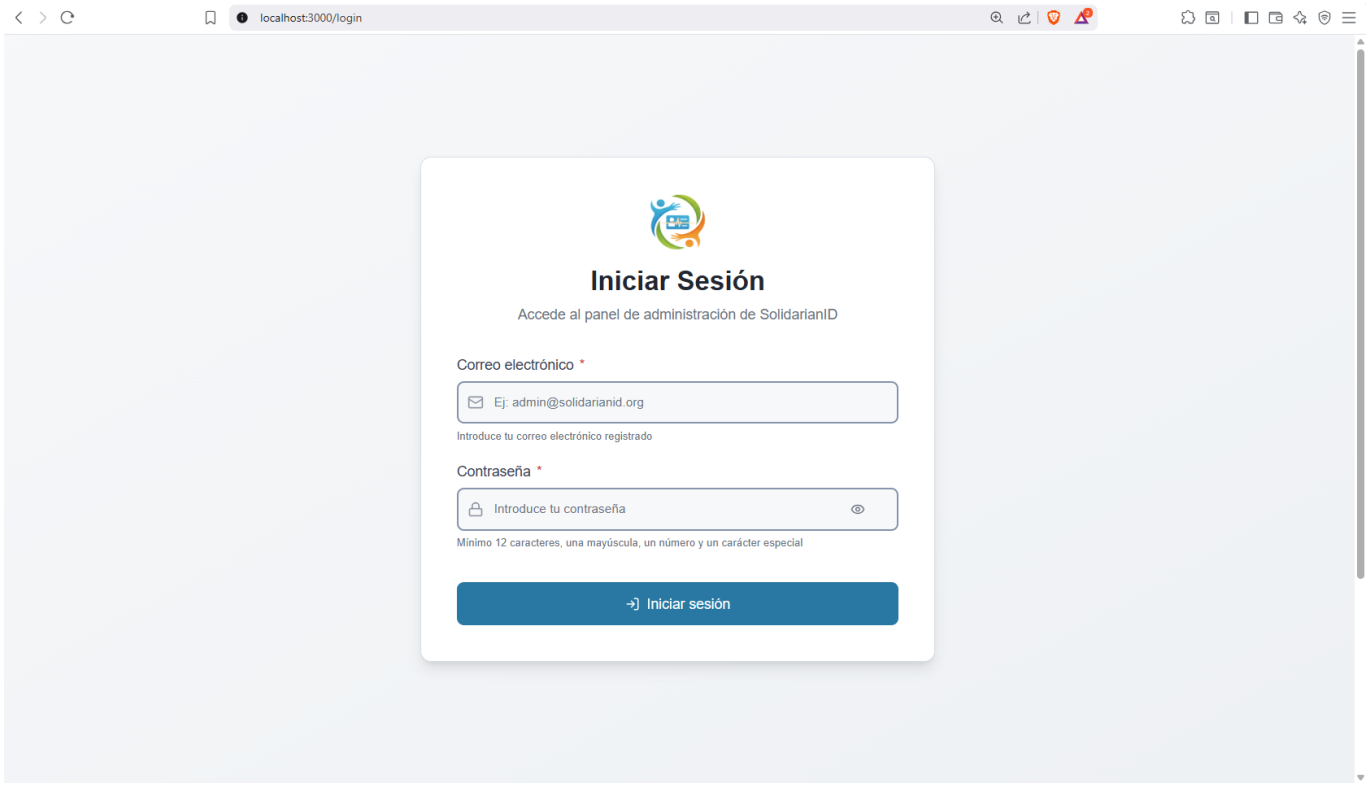


Figura 7: Pantalla de login

Registro de Administradores

El formulario de registro (`register.hbs`, Figura 8) permite la creación de nuevas cuentas administrativas. Extiende las capacidades del login incorporando:

- **Formulario extenso**: Campos para nombre, apellidos, email, contraseña, confirmación de contraseña, fecha de nacimiento (con restricción de mayoría de edad), nacionalidad (selector con flags), género y teléfono
- **Validación sincronizada**: Comprobación en tiempo real de coincidencia de contraseñas mediante JavaScript
- **Navegación contextual**: Enlace de retorno al dashboard con tooltip explicativo
- **Feedback visual**: Estados de validación por campo con iconos de éxito/error dinámicos

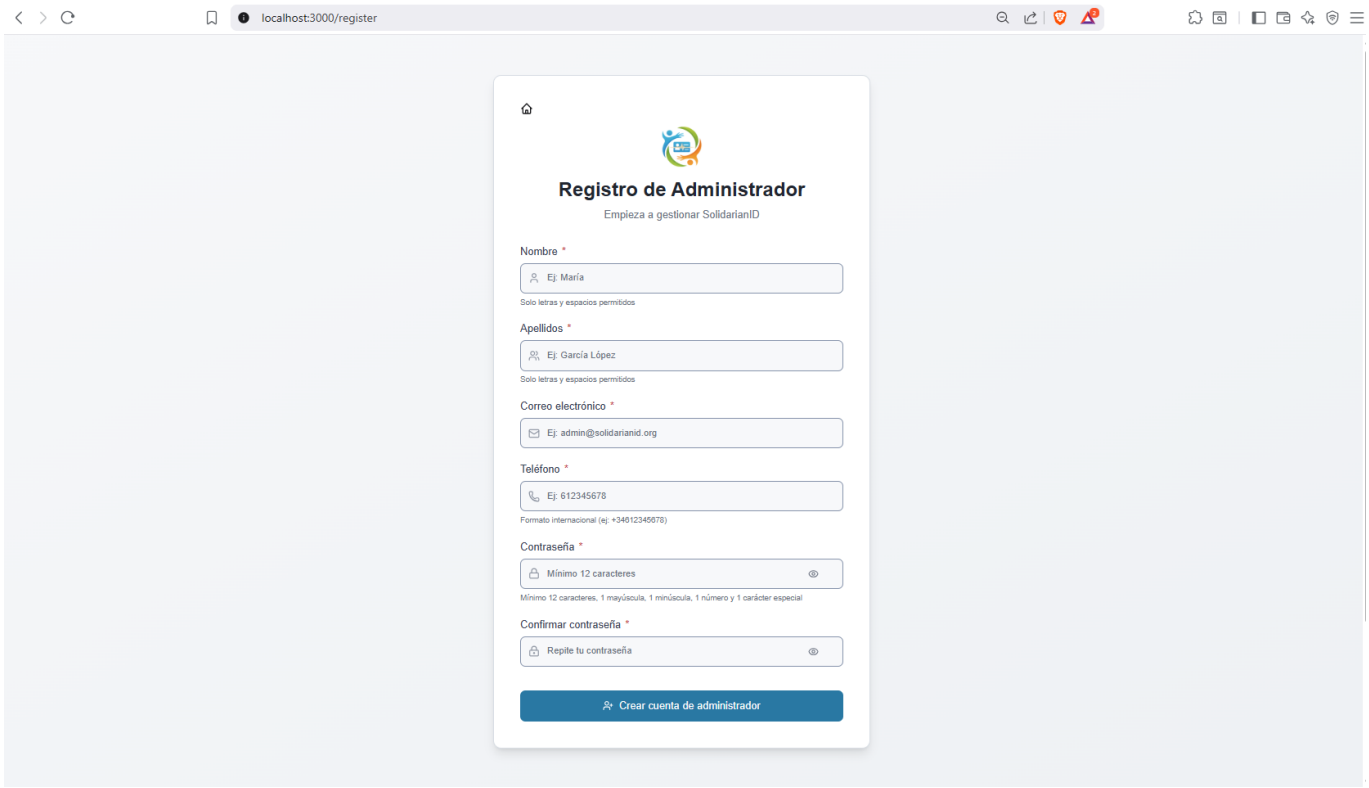


Figura 8: Pantalla de registro

Dashboard Analítico

La vista `metrics.hbs` (Figura 9) constituye el panel principal de métricas administrativas. Integra:

- **Gráficos interactivos**: Cuatro visualizaciones Chart.js (comunidades top, nacionalidades, causas por ODS, evolución temporal de adhesiones) con configuración serializada desde el backend
- **Tarjetas de resumen**: Cuatro KPIs principales (total comunidades, usuarios activos, causas, acciones) con atributos `aria-label` descriptivos y variantes cromáticas temáticas
- **Manejo de estados**: Renderizado condicional de alertas de "Sin datos" o "Error de conexión" mediante lógica JavaScript (ver `metrics.js`)
- **Soporte Dark Mode**: Detección automática de esquema de colores preferido mediante `prefers-color-scheme` que ajusta la paleta de Chart.js

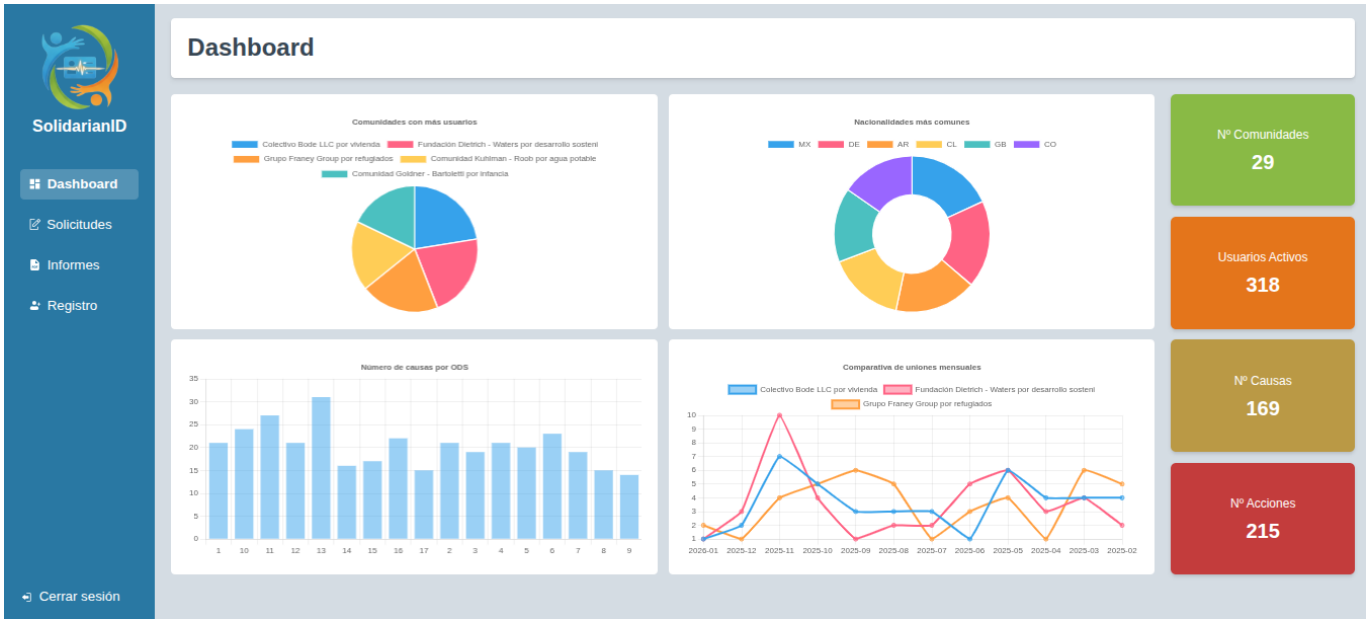


Figura 9: Panel de estadísticas de la aplicación

Sistema de Validación de Solicitudes

Las tres vistas de gestión de solicitudes (`community-requests.hbs`, `cause-requests.hbs`, `action-requests.hbs`) comparten arquitectura común basada en *partials* especializados.

Validación de Comunidades

La interfaz `community-requests.hbs` (Figura 10) permite a los administradores aprobar o rechazar solicitudes de creación de comunidades. Funcionalidades implementadas:

- **Listado paginado**: Renderizado server-side de tarjetas expandibles (`tab-community.hbs`) con contador de solicitudes pendientes
- **Acciones de validación**: Botones duales de aprobación/rechazo con confirmación modal (`reason-modal.hbs`) para justificar denegaciones
- **Información contextual**: Cada tarjeta muestra nombre de comunidad, descripción, fecha de solicitud, y datos del solicitante
- **Paginación accesible**: Componente `pagination.hbs` con navegación por teclado y anuncios ARIA



Figura 10: Panel de validación de solicitudes de comunidades

Validación de Causas Solidarias

La vista `cause-requests.hbs` (Figura 11) extiende el patrón anterior incorporando validación específica de causas benéficas. Elementos distintivos:

- **Mapeo de ODS**: Visualización explícita de los Objetivos de Desarrollo Sostenible asociados a cada causa (requisito fundamental del dominio SolidarianID)
- **Descripción extendida**: Tarjetas con mayor contenido textual (misión, objetivos, impacto esperado)
- **Validación administrativa**: Sistema de aprobación binaria con modal de justificación obligatoria para rechazos

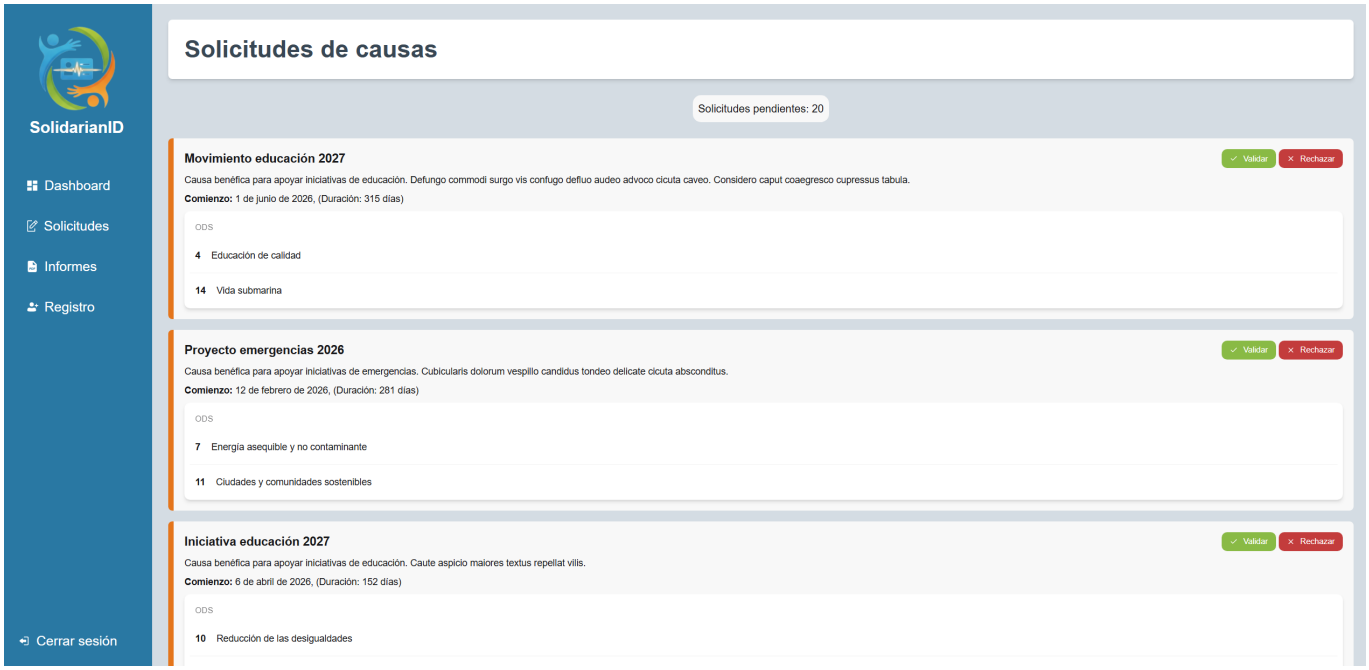


Figura 11: Panel de validación de solicitudes de causas

Validación de Acciones Caritativas

El formulario `action-requests.hbs` (Figura 12) gestiona la aprobación de acciones concretas vinculadas a causas existentes. Particularidades:

- **Objetivos cuantificables**: Cada acción especifica métricas de impacto (ej: recolectar 500kg de alimentos")
- **Temporalidad**: Campos de fecha inicio/fin y ubicación geográfica si aplica
- **Jerarquía relacional**: Referencia a la causa padre mediante breadcrumbs o badges informativos

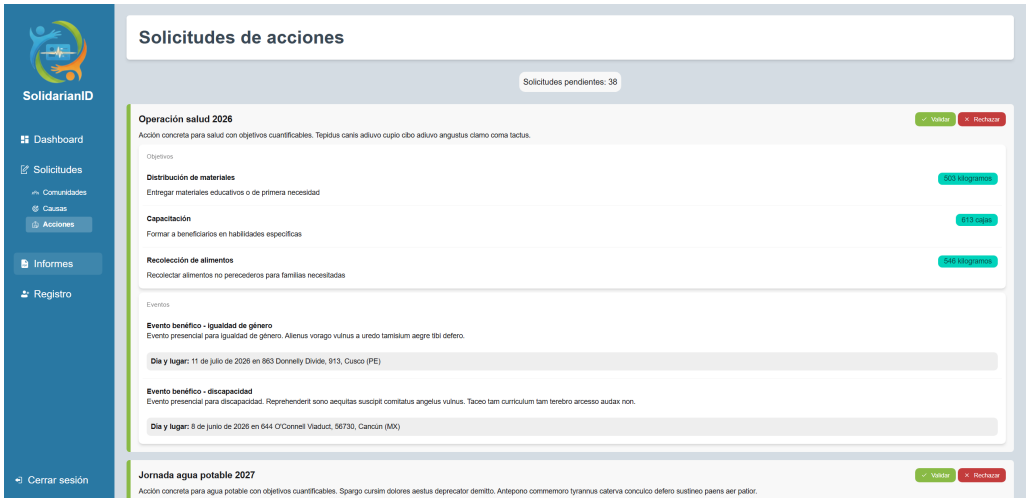


Figura 12: Panel de validación de solicitudes de acciones

Cuadro de diálogo para introducir el motivo

Este cuadro de diálogo sirve para introducir un motivo cuando el administrador valida o rechaza la solicitud de creación de una comunidad, causa o acción. Si el administrador valida la comunidad, no es necesario introducir ningún motivo, pero si la rechaza, deberá escribir un motivo de mínimo 20 caracteres. Este cuadro de diálogo se puede observar en la Figura 13.

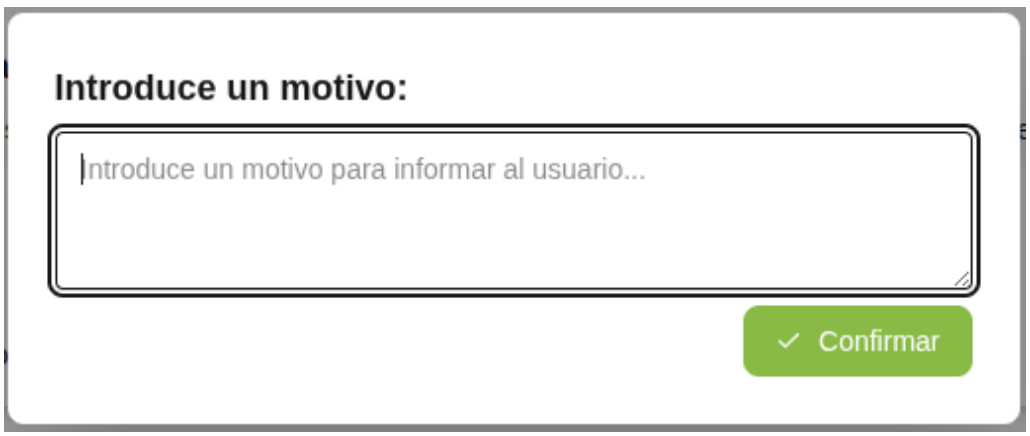


Figura 13: Cuadro de diálogo para introducir el motivo

Generación de Informes de Usuario

Listado de Usuarios

La vista `reports.hbs` (Figura 14) proporciona funcionalidad de exploración y exportación de perfiles de usuario. Implementación técnica:

- **Grid responsive**: Sistema de tarjetas adaptativo (1 columna móvil, 2 tablet, 3 desktop) con renderizado server-side para optimizar SEO y accesibilidad
- **Interactividad**: Cada tarjeta de usuario es clickeable (`role="button"`, `tabindex="0"`) y redirige a vista detallada mediante JavaScript
- **Metadatos visuales**: Muestra username, ubicación geográfica (ciudad + código país), número de comunidades asociadas
- **Paginación estática**: Generación server-side de controles de navegación con lógica de elipsis para grandes datasets
- **Estados de error**: Renderizado condicional de pantallas de vacío ("No usuarios"), error de conexión, o resultados válidos

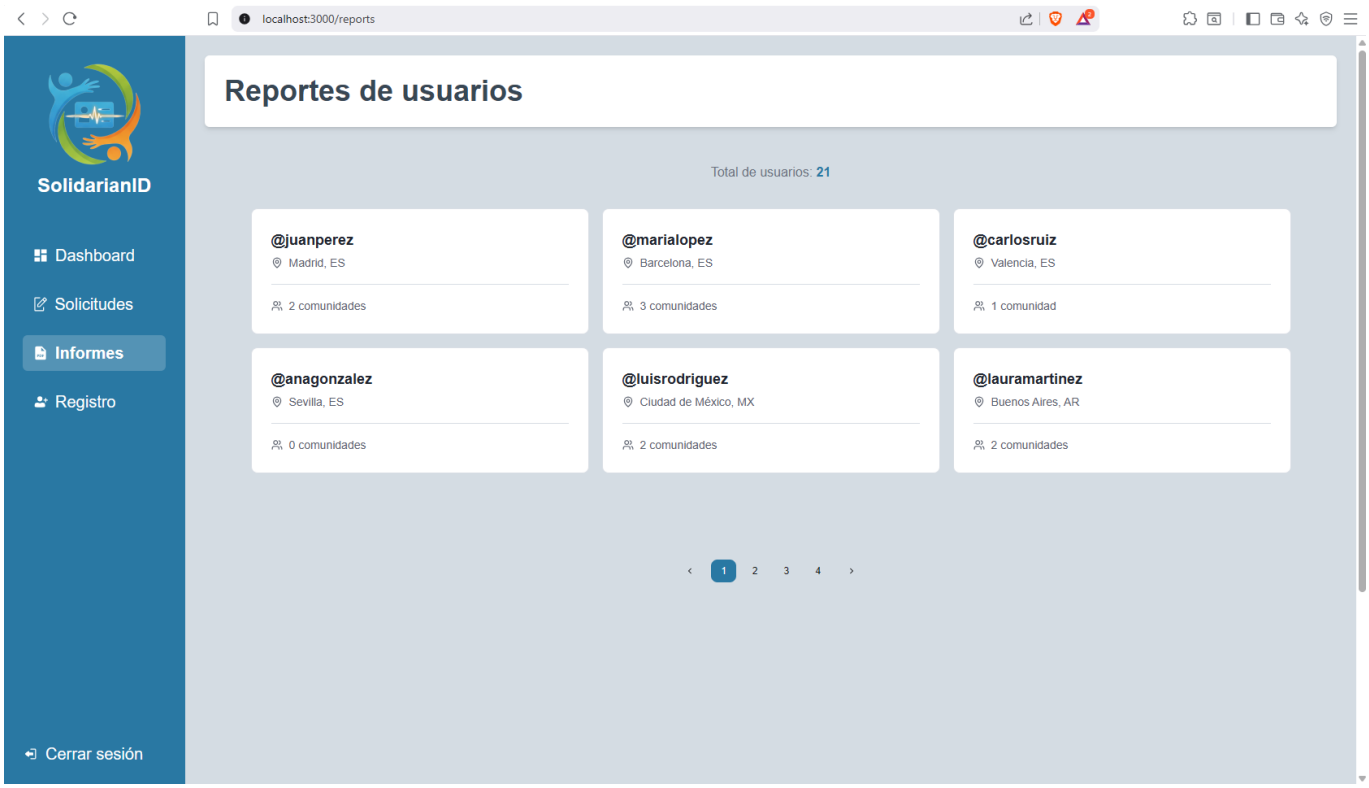


Figura 14: Panel de informes de usuario

Detalle de Usuario y Exportación PDF

Aunque la Figura 15 muestra la interfaz de detalle individual, esta vista permite:

- **Información completa**: Perfil completo del usuario incluyendo historial de participación (currículum voluntario)
- **Exportación PDF**: Generación server-side de documento PDF con el resumen de actividades del usuario mediante librería especializada (potencialmente Puppeteer o PDFKit)

- **Navegación contextual**: Breadcrumbs de retorno al listado manteniendo estado de paginación

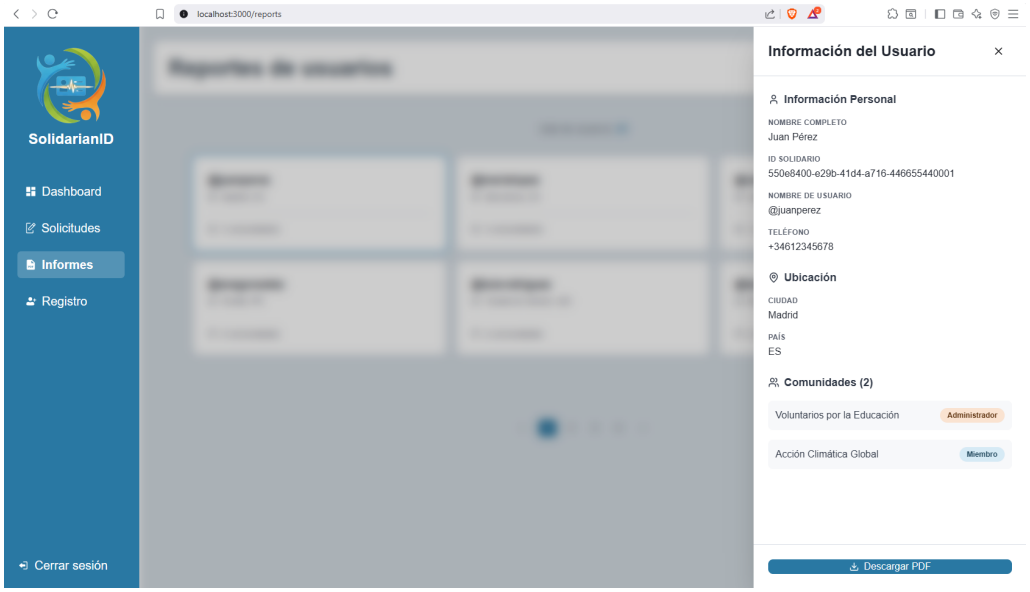


Figura 15: Vista detallada de usuario

Gestión de Errores

Página de Error Centralizada

La vista `error.hbs` (Figura 16) implementa un manejador global de errores HTTP siguiendo el patrón de NestJS ExceptionFilter. Características:

- **Códigos HTTP estándar**: Renderiza dinámicamente el código de estado (404, 500, 403, etc.) y mensaje personalizado
- **Región ARIA live**: El elemento principal tiene `role=‘‘alert’’` y `aria-live=‘‘assertive’’` para notificar inmediatamente a lectores de pantalla
- **Navegación de recuperación**: Botón primario de retorno al dashboard y botón secundario opcional al inicio público
- **Iconografía descriptiva**: Icono SVG grande centrado para refuerzo visual del estado de error
- **Focus management**: El título del error tiene `tabindex=‘‘-1’’` permitiendo establecer foco programáticamente para mejorar la experiencia de navegación asistida

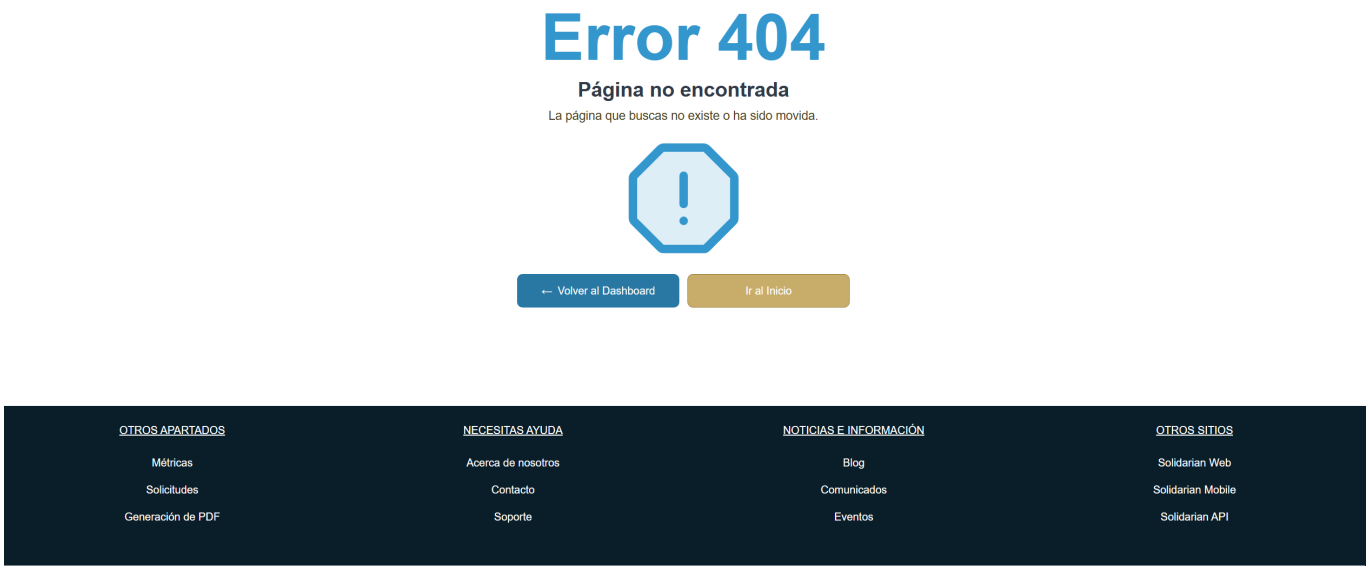


Figura 16: Página de errores con Not Found